

SMART HOSPITAL & RESOURCE ALLOCATION SYSTEM

PROJECT REPORT

Course: CSC 1012 – Introduction to Computer Programming

Programme: Bsc(General) Degree - First year

Faculty: Faculty of Applied Science

University: university of sri jayawardenapura

Assignment: individual assignment - 2026

Student Name: M.G.Shehan Rashmika Anuhas

Index Number: AS20250490

Registration Number: AS128741

Date of Submission: 20.09.2026

Github repository URL:

<https://github.com/Rash200522/Smart-Hospital-Resource-Allocation-System.git>

1.Introduction

The Smart Hospital & Resource Allocation System is a menu-driven console application developed in the C programming language. It is designed to simulate the core operations of a small hospital: patient intake, bed allocation, emergency triage, medical billing, and resource reporting.

Hospitals in real life face complex decisions daily — who gets priority, which bed is available, what should be charged, and how resources are used. This project models a simplified version of that environment using only fundamental C programming constructs.

The system was developed as an individual assignment to demonstrate competence in:

- 1D and 2D arrays (parallel arrays)
- Custom functions and modular programming
- String handling using character arrays
- Loops, conditionals, and switch statements
- Mathematical modelling for billing
- Basic file handling for data persistence

The entire system is written without using struct, following the recommended parallel arrays approach stated in the assignment specification. This choice demonstrates a strong grasp of index-based data management.

2.Project Objectives

The primary objectives of the project were:

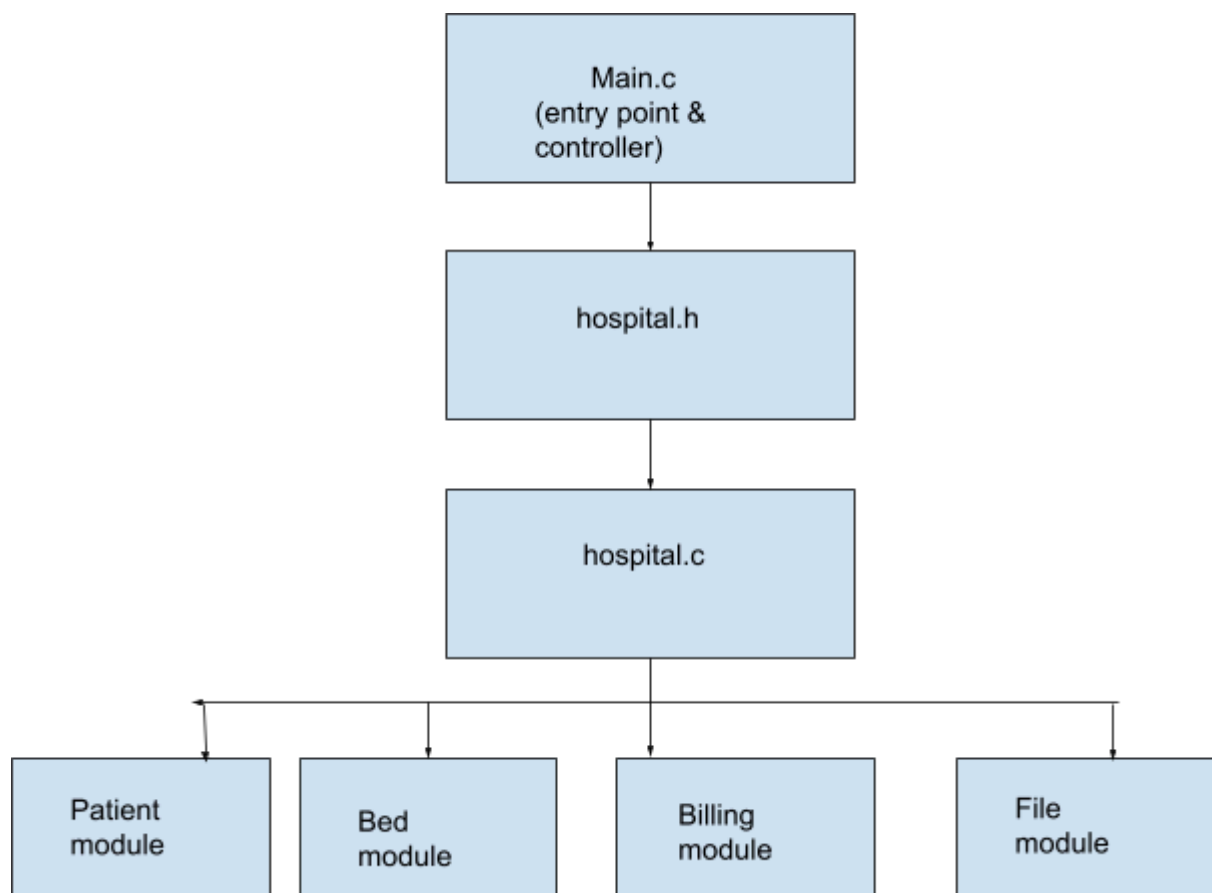
1. To design a modular, menu-driven application in C.
2. To simulate patient intake, triage, and billing workflows.
3. To manage hospital bed occupancy using a 2D array.
4. To implement priority-based sorting for emergency triage.
5. To calculate patient bills using well-defined formulas (surcharges, subsidies, ward costs).
6. To generate meaningful analytics and reports for hospital administration.
7. To persist critical data across sessions using file handling.

8. To apply good programming practices: modular functions, input validation, and defensive coding

3. High-Level Architecture

The system follows a modular architecture, where responsibilities are separated into distinct functions. The main program acts as a controller, calling functions based on user input.

Architectural Diagram



Module Responsibilities

<u>Module</u>	<u>Responsibility</u>
<u>Main Controller</u>	<u>Presents menu, accepts input, dispatches to appropriate function</u>
<u>Patient Module</u>	<u>Registration, validation, ID generation</u>
<u>Bed Module</u>	<u>2D occupancy tracking, allocation, release</u>
<u>Billing Module</u>	<u>Surcharges, ward costs, subsidies, final amounts</u>
<u>Priority Module</u>	<u>Emergency triage sorting (bubble sort)</u>
<u>Reports Module</u>	<u>Statistics, occupancy, revenue summaries</u>
<u>File Module</u>	<u>Reading/writing bed status and patient records</u>
<u>Utility Module</u>	<u>Input validation, string formatting, separators</u>

5. Implemented Functions

The system implements 30+ functions, organized by module. Each performs one well-defined task, following the good design principle discussed in lectures.

5.1 Initialization Functions

Function	Purpose
<code>ensureDataFolderExists()</code>	Creates <code>data/</code> folder if missing
<code>initializeSystem()</code>	Resets all patient and system data to safe defaults
<code>initializeBedOccupancy()</code>	Sets all beds to available

5.2 File Handling Functions

Function	Purpose
<code>loadBedStatusFromFile()</code>	Reads persisted bed status
<code>saveBedStatusToFile()</code>	Writes current bed status
<code>appendPatientRecord()</code>	Appends billing log for a patient

<code>loadPatientRecords()</code>	<u>Counts historical records at startup</u>
-----------------------------------	---

5.3 Menu & Display Functions

Function	Purpose
<code>displayMainMenu()</code>	<u>Presents primary menu</u>
<code>displaySpecialtyMenu()</code>	<u>Prints specialty lookup table</u>
<code>displayWardMenu()</code>	<u>Prints ward table with occupancy bars</u>
<code>printSeparator()</code>	<u>Prints <code>====</code> divider</u>
<code>printHeader()</code>	<u>Prints titled section header</u>

5.4 Patient Registration Functions

Function	Purpose
<code>registerNewPatient()</code>	<u>Complete intake workflow</u>
<code>generatePatientID()</code>	<u>Generates ID: <code>PAT-1001</code>, <code>PAT-1002</code>, ...</u>
<code>findAvailableBed()</code>	<u>Locates first free bed in a ward</u>
<code>assignBedToPatient()</code>	<u>Marks a bed as occupied</u>

5.5 Billing Functions

Function	Purpose
<code>calculateEmergencySurcharge()</code>	<u>25%/50% surcharge based on urgency</u>
<code>calculateAgeSubsidy()</code>	<u>15% discount for age <5 or >65</u>
<code>calculateWaitingTime()</code>	<u>Queue count × specialty time</u>
<code>displayBill()</code>	<u>Formatted bill output</u>

5.6 Sorting & Priority Functions

Function	Purpose
<code>sortPatientsByPriority()</code>	<u>Bubble sort by urgency then registration</u>
<code>displayPatientsByPriority()</code>	<u>Shows triage-ordered list</u>

5.7 Reports & Analytics Functions

Function	Purpose
<code>generateReports()</code>	<u>Reports submenu</u>
<code>displayUrgencyStatistics()</code>	<u>Counts patients per urgency level</u>
<code>displayRevenueStatistics()</code>	<u>Gross, discounts, net revenue</u>
<code>displayBedOccupancyReport()</code>	<u>Ward-by-ward occupancy table</u>
<code>displayHighestPayingPatient()</code>	<u>Finds patient with max final bill</u>

5.8 Utility Functions

Function	Purpose
<code>getValidIntInput(min, max)</code>	Reads and validates integers
<code>getValidStringInput(buf, len)</code>	Reads and validates names
<code>clearInputBuffer()</code>	Flushes <code>stdin</code> if needed

6. Assumptions Made

1. The following assumptions were made during development and are documented for transparency:
2. Patient capacity: Maximum 200 patients can be registered per session; sufficient for demonstration scope.
3. Name validation: Names are limited to 49 characters, must be non-empty, and cannot contain digits.
4. Age range: 0 to 120 years; values outside are rejected as input errors.
5. Ward admission fallback: If a selected ward is full, the patient is registered as OPD (outpatient) with ward cost 0.00 LKR.
6. Daily cap: The specialty daily cap is treated as informational — a warning is displayed but registration still proceeds.
7. Bed numbering: Beds are displayed as 1-indexed (#01–#20) for user friendliness, but stored 0-indexed internally.
8. Critical waiting time: Critical (Level 3) patients always show "Immediate Attention" regardless of queue length.
9. Queue increment: The specialty queue count is incremented after each registration, so the first patient in a specialty sees a waiting time of 0.

10. Currency: All monetary values are in Sri Lankan Rupees (LKR).
11. Time zone: File timestamps use the system's local time.
12. Patient IDs: Auto-generated in format PAT-XXXX starting from PAT-1001.
13. Data folder: Located at data/ relative to the executable; auto-created if missing.
14. Single-user: The system assumes one user at a time — no concurrency control.
15. No authentication: The system has no login; intended for demonstration only.
16. ICU capacity constraint: ICU is limited to 5 beds as per specification, so bed #06+ cannot be assigned even though bedOccupancy[3] has 20 columns.